R Programming & Statistics Cheatsheet

Based on Sessions 15–18 | MCQ & Quick Reference Guide

SESSION 15: INTRODUCTION

Basics & Environment

- **R**: Interpretive, case-sensitive language for statistical computing.
- **Assignment**: Preferred operator is `<-`.

```
x <- 5  # Standard R assignment
x = 5   # Works, but discouraged
```

- **Comments**: Start with #.
- **Help**:
  - `?function_name` (e.g., `?mean`)
  - `help("function_name")`
  - `help.search("keyword")` or `??keyword`

RStudio Interface

- **Source**: Write/edit scripts.
- **Console**: Execute code, view output.
- **Environment**: View active variables/objects.
- **Plots/Packages/Help**: Visualization and management.

Arithmetic Operators

- + Addition, - Subtraction
- * Multiplication, / Division
- ^ or ** Exponentiation
- %% Modulus (Remainder)
- %/% Integer Division

MCQ Tip: Order of operations follows PEMDAS.

```
5 %% 2  # Result: 1
5 %/% 2 # Result: 2
```

SESSION 16: DATA OBJECTS

Data Types (Atomic Vectors)

- **Numeric**: Default for numbers (e.g., 10.5, 4).
- **Integer**: Whole numbers, suffix L (e.g., 5L).
- **Character**: Text in quotes (e.g., `"Hello"`).
- **Logical**: TRUE (T) or FALSE (F).
- **Complex**: e.g., `1 + 4i`.

Check types: `class(x)`, `typeof(x)`, `str(x)`.

Data Structures

- **Vector**: 1D, same data type.

```
v <- c(1, 2, 3)
```

- **Matrix**: 2D, same data type.

```
m <- matrix(1:6, nrow=2, ncol=3)
```

- **Array**: n-dimensional, same type.
- **List**: 1D, **mixed** data types.

```
l <- list(name="John", age=25, scores=c
    (90,80))
```

- **Data Frame**: 2D, columns can have different types (like Excel).

```
df <- data.frame(ID=1:2, Name=c("A","B"))
```

Importing/Exporting Data

- **CSV**: `read.csv("file.csv")`, `write.csv(df, "file.csv")`
- **Text**: `read.table("file.txt", sep="\t")`
- **Excel**: Requires package (e.g., `readxl`).
- **Checking Data**:
  - `getwd()` : Get working directory.
  - `setwd("path")` : Set working directory.
  - `ls()` : List objects in workspace.
  - `rm(x)` : Remove object x.

Packages

- **Install**: `install.packages("pkg_name")` (Needs quotes).
- **Load**: `library(pkg_name)` (No quotes needed).

SESSION 17: MANIPULATION

Base R Manipulation

- **Indexing**: R uses **1-based indexing**.
- `vec[1]` returns 1st element.
- `df[row, col]` notation.
- **Dollar Sign $**: Access column by name.

```
df$Age  # Returns Age column as vector
```

- **Subset**:

```
subset(df, Age > 20)
```

- **Merge**: Join two data frames.

```
merge(df1, df2, by="ID")
```

- **Binding**:
  - `rbind()`: Combine rows (vertical).
  - `cbind()`: Combine columns (horizontal).

The Tidyverse (dplyr)

A collection of packages sharing common philosophy.

- **Pipe Operator (%>%)**: Passes output of left function as first argument to right function.
- **Key Verbs**:
  - `filter()`: Pick rows.
  - `select()`: Pick columns.
  - `arrange()`: Sort rows.
  - `mutate()`: Create new columns.
  - `summarise()`: Aggregate data (often with `group_by()`).

SESSION 18: FUNCTIONS

Built-in Functions

- **Math**: `abs()`, `sqrt()`, `log()`, `exp()`.

- **Stats**: `mean()`, `median()`, `sd()`, `var()`, `range()`.
- **Structure**:
  - `head(x, n)`: First n rows.
  - `tail(x, n)`: Last n rows.
  - `str(x)`: Structure of object.
  - `summary(x)`: Statistical summary.
- **Sequence**: `seq(from, to, by)`, `rep(x, times)`.

User Defined Functions

Structure of a function:

```
my_func <- function(arg1, arg2) {
  result <- arg1 + arg2
  return(result)
}
```

R Markdown

- File extension: `.Rmd`
- **YAML Header**: Metadata (title, author) between `---`.
- **Code Chunks**: Defined by ```` ```{r} ```` and ```` ``` ````.
- Used for reproducible reporting (HTML, PDF, Word).

MCQ GOTCHAS

- **Recycling**: If you add vectors of different lengths, R repeats the shorter one.
  ```
  c(1,2) + c(1,2,3,4) # Result: 2,4,4,6
  ```
- **NA vs NULL**:
  - `NA`: Missing value (holds a slot).
  - `NULL`: Does not exist (empty).
  - Check with `is.na()` or `is.null()`.
- **Coercion**: If you mix types in a vector, they convert to the most flexible type: Logic $\rightarrow$ Integer $\rightarrow$ Numeric $\rightarrow$ Character.
  ```
  c(1, "a") # Becomes "1", "a"
  ```